

SAÉ 5.01

Rendu 9 - 12/12/2025



Suivis par :

BONDON Loric

BRODIER Baptiste

SCHALLER Théo

Date de réalisation :

Année 2025/2026

Table des matières

Table des matières.....	2
I. Rappel.....	3
II. Rapport.....	3
1. Intégration complète du système d'entraînement Python.....	3
2. Architecture de la solution.....	3
3. Configuration et utilitaires développés.....	3
4. Refactorisation du service d'entraînement.....	4
5. Modifications du script Python.....	4
6. Correction des chemins de fichiers.....	4
7. Développement de la fonctionnalité galerie.....	4
8. Amélioration de l'affichage des résultats.....	5
9. Personnalisation de l'icône de l'application.....	5
10. Gestion d'erreurs robuste.....	5
11. Tests et validation.....	5
12. Résultats.....	5

I. Rappel

Sujet : Expressions du visage (dataset FER-2013)

GitHub : <https://github.com/LoricBndn/SAE5.01>

- LoricBndn = Loric BONDON
- Yoshicolors = Baptiste BRODIER
- Orphais = Théo SCHALLER

Trello : <https://trello.com/b/GYnaCKyV/sae501>

II. Rapport

1. Intégration complète du système d'entraînement Python

Cette semaine, on a finalisé l'intégration du système d'entraînement personnalisé en remplaçant la simulation par un véritable d'apprentissage automatique connectant le script Python d'entraînement au backend Spring Boot via des appels système pour permettre un entraînement réel des modèles personnalisés. Lorsque l'utilisateur lance l'entraînement depuis l'application Android, le backend charge les images depuis la base de données MySQL, les inscrit sous JSON, puis écrit ce JSON dans un fichier temporaire. Le backend lance ensuite un programme Python qui entraîne le modèle pendant quinze époques et génère un fichier TensorFlow Lite. Le backend lit ensuite ce modèle, le stocke dans la base de données, puis nettoie les fichiers temporaires.

2. Modifications du script Python

Le script Python a été modifié pour accepter un chemin de fichier JSON au lieu d'une chaîne JSON directe, ce qui évite les limitations de longueur de ligne de commande Windows. On a ajouté le support du décodage base64 pour les images reçues depuis le backend. Le système de progression a été amélioré en boucle manuelle sur les époques pour afficher après chaque époque le numéro et la précision de validation au format parsable par le backend.

3. Développement de la fonctionnalité galerie

On a commencé le développement d'une nouvelle fonctionnalité permettant d'importer une image depuis la galerie du téléphone pour appliquer la reconnaissance d'émotions dessus. Cette fonctionnalité complète la détection en temps réel via caméra en permettant aux utilisateurs d'analyser des photos existantes. Le travail est actuellement en cours et sera finalisé prochainement.

4. Amélioration de l'affichage des résultats

On travaille également sur une modale d'affichage des résultats de détection plus détaillée. Cette modale affichera l'expression reconnue avec son nom complet, le pourcentage de confiance de la prédiction, et lorsqu'un modèle personnalisé est disponible, elle indiquera si la prédiction provient du modèle de base, du modèle personnalisé ou d'une combinaison des deux. Cette interface permettra aux utilisateurs de mieux comprendre comment l'IA analyse leurs expressions et de visualiser l'apport de leur modèle personnalisé.

5. Personnalisation de l'icône de l'application

En parallèle, on a remplacé l'icône Android par défaut par une icône personnalisée. On a créé des versions PNG de l'icône dans cinq résolutions différentes correspondant aux densités d'écran Android : mdpi 48x48 pixels, hdpi 72x72, xhdpi 96x96, xxhdpi 144x144 et xxxhdpi 192x192. Ces fichiers ont été placés dans les dossiers mipmap appropriés sous les noms `ic_launcher.png` et `ic_launcher_round.png`. On a également nettoyé les anciennes ressources en supprimant les fichiers `webp` par défaut et les fichiers XML d'icônes adaptatives.



6. Gestion d'erreurs

On a implémenté une gestion complète des erreurs pour tous les scénarios possibles : Python non installé avec vérification au démarrage, bibliothèques Python manquantes détectées via l'analyse des messages d'erreur, données insuffisantes avec validation du nombre minimum d'images par catégorie, timeout d'entraînement dépassant trente minutes, et fichier modèle absent après l'entraînement. Toutes ces erreurs sont capturées et remontées à l'utilisateur via le système de statut avec des messages clairs.